

Model V5 — Hierarchical Cost-Per-Person Baseline

Model V5 extends V3 by placing a hierarchical prior on the region-specific intercept α_i :

$$\alpha_i \sim \mathcal{N}(\mu_\lambda \cdot c_i, \tau_\alpha)$$

where c_i is the HR budget per 10k population for region i (from `hr_budget_scaledPer10k.csv`), μ_λ is a global cost coefficient estimated from the data, and τ_α captures residual region-level variation not explained by cost.

The mean function is identical to V3 (additive α_i). Cost enters only through the hierarchical prior, shrinking each region's baseline toward a cost-predicted value while allowing deviations via τ_α .

```
library(rjags)

## Loading required package: coda
## Linked to JAGS 4.3.2
## Loaded modules: basemod,bugs

library(dplyr)

##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(tibble)
library(stringr)
library(ggplot2)
library(tidyr)

output_folder <- "../data/models/v5/"
save_output   <- TRUE
if (save_output) dir.create(output_folder, recursive = TRUE, showWarnings = FALSE)
```

Load Data

```
uecDf    <- read.csv("../data/wide_weekly_scaledPer10k.csv")
regions  <- uecDf$Region
n.region <- length(regions)
uecDf    <- uecDf %>% select(-Region)
n.weeks  <- length(names(uecDf))
uecMat   <- as.matrix(uecDf)
```

Cost Per Person

```
cpp_df <- read.csv('../data/hseBudget_scaledPer10k.csv')

# Align to region order from uecDf
budget_scaledPer10k_raw <- cpp_df$BudgetPer10k[match(regions, cpp_df$Region)]

# Rescale: divide by 1000 so units = thousands of euro per 10k population
# This puts mu_lambda on an interpretable scale and improves MCMC mixing
budget_scaledPer10k <- budget_scaledPer10k_raw / 1000

data.frame(Region = regions,
            BudgetPer10k_raw = round(budget_scaledPer10k_raw, 4),
            BudgetPer10k_k = round(budget_scaledPer10k, 4))
```

```
##              Region BudgetPer10k_raw BudgetPer10k_k
## 1 HSE Dublin and Midlands      34055.93      34.0559
## 2 HSE Dublin and North East    28599.54      28.5995
## 3 HSE Dublin and South East    27381.52      27.3815
## 4           HSE Mid West      28615.77      28.6158
## 5           HSE South West     27031.63      27.0316
## 6 HSE West and North West     33067.77      33.0678
```

Model V4 Specification

Hierarchical prior (V4 key change):

$$\alpha_i \sim \mathcal{N}(\mu_\lambda \cdot c_i, \tau_\alpha)$$

```
model <-
"
model{
  for(i in 1:I){
    #-----
    # Likelihood
    #-----
    y[i,1] ~ dnorm(mu[i,1], tau[i])
    for(t in 2:T){
      y[i,t] ~ dnorm(mu[i,t] + (phi * (y[i,t-1] - mu[i,t-1])), tau[i])
    }

    # Mean function (same as V3)
    for(t in 1:T){
      mu[i,t] <- alpha[i] +
        beta[i]      * cos((2 * pi) * (t/52)) +
        gamma[i]     * sin((2 * pi) * (t/52)) +
        delta_pre[i] * ny_pre[t] +
        delta_mid[i] * ny_mid[t] +
        delta_post[i] * ny_post[t] +
        sigma_pre    * fr_pre[t] * mw[i] +
        sigma_mid    * fr_mid[t] * mw[i] +
        sigma_post   * fr_post[t] * mw[i]
    }
    # Hierarchical prior: alpha[i] shrinks toward cost-predicted baseline
    alpha[i] ~ dnorm(mu_lambda[i] * budget_scaledPer10k[i], tau_alpha)
    # Per-region cost coefficient with partial pooling toward global mean
```

```

    mu_lambda[i] ~ dnorm(mu_global, tau_mu)
    beta[i] ~ dnorm(0, 0.001)
    gamma[i] ~ dnorm(0, 0.001)
    tau[i] ~ dgamma(0.001, 0.001)
    delta_pre[i] ~ dnorm(0, 0.001)
    delta_mid[i] ~ dnorm(0, 0.001)
    delta_post[i] ~ dnorm(0, 0.001)
  }
  phi ~ dunif(-1, 1)
  sigma_pre ~ dnorm(0, 0.001)
  sigma_mid ~ dnorm(0, 0.001)
  sigma_post ~ dnorm(0, 0.001)
  # Hyperpriors
  mu_global ~ dnorm(0, 0.01) # global cost coefficient (SD = 10)
  tau_mu ~ dgamma(0.001, 0.001) # precision of per-region cost deviations
  tau_alpha ~ dgamma(0.001, 0.001)
}
"

# Event indicators
week_mod <- (1:n.weeks) %% 52
ny_pre <- as.numeric(week_mod == 0)
ny_mid <- as.numeric(week_mod == 1)
ny_post <- as.numeric(week_mod == 2)
fr_pre <- as.numeric((1:n.weeks) == 86)
fr_mid <- as.numeric((1:n.weeks) == 87)
fr_post <- as.numeric((1:n.weeks) == 88)
mw <- as.numeric(regions == "HSE Mid West")

```

Fit

```

exp_jags <- jags.model(
  textConnection(model),
  data = list(
    y = uecMat,
    I = n.region,
    T = n.weeks,
    pi = pi,
    budget_scaledPer10k = budget_scaledPer10k,
    ny_pre = ny_pre,
    ny_mid = ny_mid,
    ny_post = ny_post,
    fr_pre = fr_pre,
    fr_mid = fr_mid,
    fr_post = fr_post,
    mw = mw
  ),
  n.chains = 4
)

```

```

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes

```

```

## Graph information:
##   Observed stochastic nodes: 906
##   Unobserved stochastic nodes: 55
##   Total graph size: 6050
##
## Initializing model
update(exp_jags, n.iter = 10000) # burn-in

model_parameters <- c(
  "alpha", "beta", "gamma", "tau", "phi",
  "delta_pre", "delta_mid", "delta_post",
  "sigma_pre", "sigma_mid", "sigma_post",
  "mu_lambda", "mu_global", "tau_mu", "tau_alpha"
)

exp_sim <- coda.samples(
  model          = exp_jags,
  variable.names = model_parameters,
  n.iter         = 30000
)

```

Convergence

```

gelman_results <- gelman.diag(exp_sim, multivariate = FALSE)
gelman_results

```

```

## Potential scale reduction factors:
##
##               Point est. Upper C.I.
## alpha[1]         1.00      1.00
## alpha[2]         1.00      1.00
## alpha[3]         1.00      1.00
## alpha[4]         1.00      1.00
## alpha[5]         1.00      1.00
## alpha[6]         1.00      1.00
## beta[1]          1.00      1.00
## beta[2]          1.00      1.00
## beta[3]          1.00      1.00
## beta[4]          1.00      1.00
## beta[5]          1.00      1.00
## beta[6]          1.00      1.00
## delta_mid[1]     1.00      1.00
## delta_mid[2]     1.00      1.00
## delta_mid[3]     1.00      1.00
## delta_mid[4]     1.00      1.00
## delta_mid[5]     1.00      1.00
## delta_mid[6]     1.00      1.00
## delta_post[1]    1.00      1.00
## delta_post[2]    1.00      1.00
## delta_post[3]    1.00      1.00
## delta_post[4]    1.00      1.00
## delta_post[5]    1.00      1.00
## delta_post[6]    1.00      1.00

```

```
## delta_pre[1]      1.00      1.00
## delta_pre[2]      1.00      1.00
## delta_pre[3]      1.00      1.00
## delta_pre[4]      1.00      1.00
## delta_pre[5]      1.00      1.00
## delta_pre[6]      1.00      1.00
## gamma[1]          1.00      1.00
## gamma[2]          1.00      1.00
## gamma[3]          1.00      1.00
## gamma[4]          1.00      1.00
## gamma[5]          1.00      1.00
## gamma[6]          1.00      1.00
## mu_global         1.00      1.00
## mu_lambda[1]      1.00      1.00
## mu_lambda[2]      1.00      1.00
## mu_lambda[3]      1.00      1.00
## mu_lambda[4]      1.00      1.00
## mu_lambda[5]      1.00      1.00
## mu_lambda[6]      1.00      1.00
## phi               1.00      1.00
## sigma_mid         1.00      1.00
## sigma_post        1.00      1.00
## sigma_pre         1.00      1.00
## tau[1]            1.00      1.00
## tau[2]            1.00      1.00
## tau[3]            1.00      1.00
## tau[4]            1.00      1.00
## tau[5]            1.00      1.00
## tau[6]            1.00      1.00
## tau_alpha         1.00      1.01
## tau_mu            1.01      1.01
```

```
if (save_output) {
  write.csv(
    as.data.frame(gelman_results$psrf) %>% rownames_to_column("param"),
    paste0(output_folder, "gelman.csv"),
    row.names = FALSE
  )
}
```

DIC

```
dic_v4 <- dic.samples(exp_jags, n.iter = 20000)
dic_v4
```

```
## Mean deviance: 2278
## penalty 46.78
## Penalized deviance: 2325
```

```
df_dic <- data.frame(
  deviance = sum(dic_v4$deviance),
  penalty = sum(dic_v4$penalty),
  DIC = sum(dic_v4$deviance) + sum(dic_v4$penalty)
)
df_dic
```

```
##   deviance  penalty      DIC
## 1 2277.994 46.78446 2324.778

if (save_output) write.csv(df_dic, paste0(output_folder, "dic.csv"), row.names = FALSE)
```

Save

```
if (save_output) {
  samples_with_chain = do.call(rbind, lapply(seq_along(exp_sim), function(ch) {
    df = as.data.frame(exp_sim[[ch]])
    df$chain = ch
    df
  }))
  write.csv(samples_with_chain,
            paste0(output_folder, "raw_samples.csv"),
            row.names = FALSE)
}
```